HI TECH GROUP

Telephony Ingest API

Integration reference for CentroVision

Version 1.0 | HRX HRMS | hrx.hitechgroup.in

This document specifies the API that HRX publishes for CentroVision to send call data into. It is generated from the running implementation: every status code, error code, required field and example below is enforced by the live endpoint.

Confidential. Issued to CentroVision for the purpose of integrating with HRX.

Contents

1. Overview 3

Two channels . . . 3

2. Authentication 4

Computing the signature . . . 4

Clock skew, replay and source address . . . 4

3. Endpoints 5

3.1 GET /health . . . 5

3.2 POST /events . . . 6

3.3 Example: a manual handset dial (UAT-05) . . . 9

3.4 POST /events/batch . . . 10

3.5 POST /recordings/ready . . . 10

3.6 POST /agent-sessions . . . 12

3.7 POST /cdr/import . . . 12

4. Event catalogue (push) 14

Call lifecycle . . . 14

Transfers, conferences and forwarding . . . 14

Queue and IVR . . . 14

Recordings and agent activity . . . 15

5. Identifying the agent 16

The agent block . . . 16

6. Data dictionary 17

timestamps . . . 17

durations . . . 17

Optional but valuable . . . 17

7. Response codes and errors 19

Error body . . . 19

Error codes you are most likely to see . . . 19

8. Reliability rules 20

9. Reconciliation (pull) 21

What CentroVision needs to expose . . . 21

10. Acceptance checklist 22

Questions . . . 22

1. Overview

When a call happens on the CentroVision gateway, send a small JSON object describing it to the HRX endpoint. That is the entire integration. HRX stores, normalises and reports on the data; CentroVision is not asked to build reports, storage or any user interface.

Base URL

```
https://hrx.hitechgroup.in/api/telephony/v1
```

There is no separate test host. Integrating against the live endpoint is safe: until a key is issued the endpoint refuses every request, and once issued, test events land in a call log that starts empty and affects nothing else in the HR system.

Two channels

The integration uses webhook push as the primary channel, with a scheduled pull as a safety net. Push alone is fast but lossy: one network fault and those calls are gone with nothing to indicate they existed. Pull alone is complete but blind: no live call popup and no disposition prompt at hangup. Together they give real-time behaviour plus a daily proof that both systems agree.

Channel	Direction	When	Carries
PUSH (primary)	CentroVision to HRX	As each event happens	Call lifecycle, transfers, conferences, forwards, queue events, recordings, agent sessions
PULL (audit)	HRX to CentroVision	Nightly at 02:00 IST	Full CDR for the previous day, to detect and backfill anything the push channel never delivered

The requirement that matters most

Every call must be reported, including calls dialled manually on the handset or SIM - not only calls originated through an API request. Inbound, IVR, predictive, callback and forwarded calls must all raise the same events, with the "origin" field set correctly. A manual dial sends exactly the same payload with origin set to manual_dial and ref_id null. This is acceptance test UAT-05 and it is a go/no-go item.

2. Authentication

Every request carries four headers. Three prove identity and integrity; the fourth is optional but recommended.

Header	Value	Required
`Authorization`	Bearer <api_key>	Yes
`X-HRX-Timestamp`	Unix time in seconds (integer, not milliseconds)	Yes
`X-HRX-Nonce`	A value unique to this request. A UUID is ideal.	Yes
`X-HRX-Signature`	sha256=<lowercase hex HMAC>	Yes
`Idempotency-Key`	The event's event_id	Recommended
`Content-Type`	application/json	Yes

Computing the signature

```
signable  = timestamp + "." + nonce + "." + raw_request_body
signature = "sha256=" + lowercase_hex( HMAC_SHA256(shared_secret, signable) )
```

Three rules

1. Sign the exact bytes you are about to send. Build the JSON string once, sign that string, transmit that string. Do not serialise, sign, then serialise again - key order or whitespace will differ by a character and every signature will fail. This is the single most common cause of a rejected request.

2. For GET /health the body is an empty string, so the signable value ends with a dot: "1755691200.abc-123." Note the trailing dot.

3. Use a fresh nonce on every HTTP request, including retries of the same event. HRX remembers nonces for 10 minutes and refuses reuse.

Worked example

```
shared_secret : s3cr3t
timestamp     : 1755691200
nonce         : 4f8c1a2e-6b3d-4e7a-9c15-2d8f0b6a1e93
body          : {"a":1}

signable      : 1755691200.4f8c1a2e-6b3d-4e7a-9c15-2d8f0b6a1e93.{"a":1}
```

Clock skew, replay and source address

Control	Rule	Refusal
Clock skew	Timestamp must be within 300 seconds of HRX server time	`stale_timestamp`
Replay	A nonce is remembered for 10 minutes	`replayed_nonce`
Source IP	Optional allowlist, set per credential	`ip_not_allowed`
Body size	Maximum 5 MB per request	`payload_too_large`
Rate	600 requests per minute per source address	`HTTP 429`

HRX compares signatures in constant time. Please do the same when verifying anything at your end.

3. Endpoints

Method and path	Purpose	Success
`GET /health`	Verify credentials, clock and source address	200
`POST /events`	Report one call event. The main endpoint.	202
`POST /events/batch`	Drain a backlog after an outage. Max 500 events.	207
`POST /recordings/ready`	Notify that audio is on disk	202
`POST /agent-sessions`	Agent login, state, break, logout	202
`POST /cdr/import`	Bulk CDR replay to close a reconciliation gap	202

3.1 GET /health

Start here. A 200 proves four things at once: the API key is correct, the shared secret is correct, your clock is within 300 seconds of ours, and your source address is permitted. Do not begin sending call data until this passes - every other failure is easier to diagnose once it does.

> The request body is empty, so the signable string ends with a dot.

Request

```
GET /api/telephony/v1/health HTTP/1.1
Host: hrx.hitechgroup.in
Authorization: Bearer hrx_9bf004780caeb3bb25c8f81284a8adcc
X-HRX-Timestamp: 1755691200
X-HRX-Nonce: 4f8c1a2e-6b3d-4e7a-9c15-2d8f0b6a1e93
X-HRX-Signature: sha256=6b2f1c...e9a4
Accept: application/json
```

Response 200

```
{
  "status": "ok",
  "server_time": "2026-08-20T16:04:22+05:30",
  "vendor": "centrovision",
  "api_version": "v1"
}
```

> Compare server_time against your own clock if you ever receive stale_timestamp.

3.2 POST /events

The main endpoint. Send one event per request, as it happens.

Target	Latency
Live lifecycle events	Within 5 seconds of occurring
call.completed	Within 30 seconds of hangup
recording.ready	Within 5 minutes of the audio being written

call.completed is the important one - it carries the full CDR and is what every report is built from. The lifecycle events before it drive live screens. If only one event per call can be delivered, deliver call.completed; a live agent popup then becomes impossible but the reporting still works.

Required parameters

Parameter	Type	Description
`event`	string	Event name from the catalogue in section 4
`event_id`	uuid	Unique for all time. This is the idempotency key - keep it STABLE across your retries, or a retry writes a second copy.
`occurred_at`	ISO-8601	When the event happened on the gateway, not when it was sent. Must carry an explicit offset, e.g. 2026-08-19T15:01:44+05:30
`call_uid`	string	Your call identifier. Must equal the RID/UID shown on the CentroVision CDR screen, so both sides can open the same record in a dispute.

Key optional parameters

Parameter	Type	Description
`sequence`	integer	Monotonically increasing within one call_uid. Lets HRX tolerate out-of-order delivery - a late call.answered will never overwrite a call.completed that already arrived.
`ref_id`	uuid|null	The HRX correlation id, supplied on originate and echoed back on every event for that call. Null for manual, inbound and IVR calls - that is expected, not an error.
`origin`	enum	api_click_to_call, manual_dial, inbound, ivr, predictive, preview, progressive, callback, internal
`direction`	enum	inbound, outbound, internal
`mode`	enum	auto, manual - the CentroVision dialling mode
`agent`	object	See section 5. This is how HRX identifies the employee.
`status`	enum	answered, no_answer, busy, failed, congestion, cancelled, abandoned, rejected, not_reachable, switched_off
`hangup_by`	enum	agent, customer, system, supervisor, timeout
`timestamps`	object	See section 6
`durations`	object	See section 6
`disposition`	object	code, name, sub_code, sub_name, remarks, callback_at
`recording`	object	available, recording_uid, duration_seconds, format, channels

Example: the smallest valid event

```
{
  "event":       "call.completed",
  "event_id":    "9f1b2c3d-4e5f-4a6b-8c7d-0e1f2a3b4c5d",
  "occurred_at": "2026-08-19T15:01:44+05:30",
  "call_uid":    "CV-20260819-000512"
}
```

Example: a complete call.completed

```
{
  "event": "call.completed",
  "event_id": "9f1b2c3d-4e5f-4a6b-8c7d-0e1f2a3b4c5d",
  "occurred_at": "2026-08-19T15:01:44+05:30",
  "sequence": 6,
  "call_uid": "CV-20260819-000512",
  "ref_id": null,
  "origin": "manual_dial",
  "direction": "outbound",
  "mode": "manual",
  "agent": {
    "agent_id": "AG-1042", "agent_code": "mehak",
    "agent_name": "Mehak Bhatia", "extension": "1042",
    "agent_number": "+919812345678", "hrms_employee_id": "EMP-096"
  },
  "caller_number": "+919812345678",
  "callee_number": "+919812309876",
  "did_number": "+911244567890",
  "campaign": { "campaign_id": "CMP-07", "campaign_name": "Customer Care",
                "queue": "CustomerCare", "skill": "Hindi" },
  "gateway":  { "gateway_id": "GW-2", "channel": "12",
                "sim_msisdn": "+919812300001", "operator": "Airtel" },
  "status": "answered",
  "hangup_by": "customer",
  "hangup_cause": "NORMAL_CLEARING",
  "hangup_cause_code": 16,
  "timestamps": {
    "initiated_at":     "2026-08-19T14:58:02+05:30",
    "agent_ring_at":    "2026-08-19T14:58:03+05:30",
    "agent_answer_at":  "2026-08-19T14:58:08+05:30",
    "customer_ring_at": "2026-08-19T14:58:10+05:30",
    "answered_at":      "2026-08-19T14:58:22+05:30",
    "ended_at":         "2026-08-19T15:01:40+05:30",
    "wrapup_start_at":  "2026-08-19T15:01:40+05:30",
    "wrapup_end_at":    "2026-08-19T15:01:44+05:30"
  },
  "durations": {
    "agent_ring_seconds": 5,  "customer_ring_seconds": 12,
    "pdd_seconds": 8,         "queue_wait_seconds": 0,
    "talk_seconds": 198,      "hold_seconds": 0,
    "wrap_seconds": 4,        "total_seconds": 218,
    "billed_seconds": 240
  },
  "disposition": {
    "code": "NI", "name": "Not Interested",
    "sub_code": "NI-PRICE", "sub_name": "Price too high",
    "remarks": "Asked to be called after Diwali"
  },
  "recording": {
    "available": true, "recording_uid": "REC-20260819-000512",
    "duration_seconds": 198, "format": "wav", "channels": "stereo"
  }
}
```

Response 202 - accepted

```
{
  "status": "accepted",
  "event_id": "9f1b2c3d-4e5f-4a6b-8c7d-0e1f2a3b4c5d",
  "received_at": "2026-08-19T15:01:45+05:30"
}
```

202 means STORED, not processed. Normalisation happens afterwards. Do not retry a 202.

Response 200 - duplicate

```
{
  "status": "duplicate",
  "event_id": "9f1b2c3d-4e5f-4a6b-8c7d-0e1f2a3b4c5d",
  "received_at": "2026-08-19T15:01:47+05:30"
}
```

This event_id was already delivered and nothing was written twice. This is the expected, successful answer to a retry - treat it as success, not as an error.

3.3 Example: a manual handset dial (UAT-05)

The agent dialled on the handset. HRX never placed the call, so there is no ref_id, and the agent block may carry only an extension - often all a handset dial knows. Everything else is identical to an API-originated call.


```
{
  "event": "call.completed",
  "event_id": "1a2b3c4d-5e6f-4071-8213-495a6b7c8d9e",
  "occurred_at": "2026-08-19T16:22:10+05:30",
  "sequence": 4,
  "call_uid": "CV-20260819-000640",
  "ref_id": null,
  "origin": "manual_dial",
  "direction": "outbound",
  "mode": "manual",
  "agent": { "extension": "1042" },
  "caller_number": "+919812345678",
  "callee_number": "+919811122233",
  "status": "no_answer",
  "hangup_by": "timeout",
  "timestamps": {
    "initiated_at":     "2026-08-19T16:21:40+05:30",
    "customer_ring_at": "2026-08-19T16:21:44+05:30",
    "ended_at":         "2026-08-19T16:22:10+05:30"
  },
  "durations": { "customer_ring_seconds": 26, "talk_seconds": 0,
                 "total_seconds": 30 }
}
```


Why this case decides the project

Integrations of this kind usually fail because only the API-originated path is instrumented, while a large share of calls are dialled manually. The reports are then wrong in week one and confidence does not recover. If the CentroVision platform cannot raise webhooks for manual, inbound and IVR calls, please say so at kickoff rather than at acceptance testing.

3.4 POST /events/batch

Backlog drain only, never live traffic. If HRX was unreachable and events were buffered, replay them here rather than one at a time. At most 500 events per request; each is processed independently, so one malformed row never rejects the batch.

Request

```
{
  "events": [
    { "event": "call.completed", "event_id": "...", "occurred_at": "...",
      "call_uid": "CV-20260819-000512", "...": "..." },
    { "event": "call.completed", "event_id": "...", "occurred_at": "...",
      "call_uid": "CV-20260819-000513", "...": "..." }
  ]
}
```

Response 207 - per item

```
{
  "accepted": 498,
  "rejected": 2,
  "results": [
    { "event_id": "9f1b...", "status": "accepted",  "error": null },
    { "event_id": "2c4d...", "status": "duplicate", "error": null },
    { "event_id": null,      "status": "rejected",
      "error": "event_id and call_uid are required" }
  ]
}
```

Always inspect results. A 207 does not mean everything was accepted.

3.5 POST /recordings/ready

Sent once the recording is written and fetchable. Send only a recording_uid - never a playable URL. When a permitted HRX user presses play, HRX requests a short-lived signed URL from CentroVision and streams the bytes itself, so audio never leaves the gateway unauthenticated and every playback is permission checked and logged. A URL in a webhook is a URL in a log file.

```
{
  "event": "recording.ready",
  "event_id": "5e6f7081-92a3-44b5-c6d7-8e9f0a1b2c34",
  "occurred_at": "2026-08-19T15:02:10+05:30",
  "call_uid": "CV-20260819-000512",
  "recording_uid": "REC-20260819-000512",
  "recording_type": "call",
  "duration_seconds": 198,
  "format": "wav",
  "channels": "stereo",
  "size_bytes": 3168000,
  "retention_until": "2027-08-19"
}
```

Parameter	Notes
`recording_uid`	Required. The handle HRX later exchanges for a signed URL.
`recording_type`	call, transfer_leg, conference or voicemail. Only "call" flips the play button on the parent call; a transfer-leg or conference recording belongs to its own leg.
`channels`	Stereo strongly preferred - agent and customer on separate channels makes talk-over analysis possible later. Mono is accepted.

Parameter	Notes
`retention_until`	If retention is finite, send it. HRX then stops offering playback after that date rather than showing a broken player.

3.6 POST /agent-sessions

This is what produces total login time and active time on the HRX productivity report. Send the CentroVision-computed totals on agent.logout: HRX stores them verbatim rather than deriving them from the state stream, specifically so that the HRX report and the CentroVision User Session report cannot drift apart and turn every discrepancy into a support ticket.

```
{
  "event": "agent.logout",
  "event_id": "6f708192-a3b4-45c6-d7e8-9f0a1b2c3d45",
  "occurred_at": "2026-08-19T19:02:11+05:30",
  "session_id": "SES-20260819-1042-01",
  "agent": { "agent_id": "AG-1042", "extension": "1042",
             "hrms_employee_id": "EMP-096" },
  "login_at":  "2026-08-19T09:58:40+05:30",
  "logout_at": "2026-08-19T19:02:11+05:30",
  "mode": "manual",
  "totals": {
    "login_seconds": 32611, "ready_seconds": 14980,
    "talk_seconds": 9120,   "hold_seconds": 340,
    "wrap_seconds": 2210,   "idle_seconds": 3105,
    "break_seconds": 2856,  "occupancy_percent": 41.2,
    "calls_handled": 63,    "calls_missed": 4
  },
  "breaks": [
    { "reason": "Lunch", "from": "2026-08-19T13:30:00+05:30",
      "to": "2026-08-19T14:05:12+05:30", "seconds": 2112 }
  ],
  "device": { "type": "gsm_handset", "ip": "192.168.2.87" }
}
```

Event	Purpose
`agent.login`	Session opens. session_id is required.
`agent.state`	Live state change: ready, oncall, hangup, wrapup, idle, break, offline, dialing, paused. Drives the wallboard.
`agent.break.start`	Break begins. Include a reason - it maps to the Offtime report.
`agent.break.end`	Break ends.
`agent.logout`	Session closes. Send the totals block here.

3.7 POST /cdr/import

Used to close a gap that the nightly reconciliation found - after a long outage, or when a webhook was never delivered. Records need no event_id: HRX derives a stable one from call_uid, so replaying the same CDR row twice collapses to a single event rather than duplicating it. Only call_uid is strictly required, but send the full CDR shape; a partial record produces a partial call row.

```
{
  "records": [
    { "call_uid": "CV-20260819-000512", "origin": "manual_dial",
      "direction": "outbound", "status": "answered",
      "agent": { "extension": "1042" },
      "timestamps": { "initiated_at": "2026-08-19T14:58:02+05:30",
                      "ended_at":     "2026-08-19T15:01:40+05:30" },
      "durations":  { "talk_seconds": 198, "total_seconds": 218 } }
  ]
}
```

```
{ "inserted": 14, "updated": 3, "rejected": 0 }
```

Prefer push. Use this to repair, not as a substitute.

4. Event catalogue (push)

Send every event the platform raises. HRX tolerates missing events but each one it does not receive is a column that stays empty on a report.

Call lifecycle

Event	When it fires	What it feeds in HRX
`call.originated`	Dial accepted	Live tile, pending call row
`call.agent_ringing`	Agent leg starts ringing	Agent ring time, agent no-answer rate
`call.agent_answered`	Agent picks up	Agent answer time
`call.customer_ringing`	Customer leg starts ringing	Ringing time, post-dial delay
`call.answered`	Both legs bridged	Talk time starts, live popup
`call.hold / call.unhold`	Agent holds or resumes	Hold analysis
`call.mute / call.unmute`	Agent mutes or unmutes	Hold analysis
`call.completed`	Hangup and wrap-up complete	The CDR row - every field in section 6. This is the most important event.

Transfers, conferences and forwarding

Event	When it fires	What it feeds in HRX
`call.transfer.initiated`	Transfer begins	Transfer report
`call.transfer.completed`	Transfer connects	Transfer report, TL recording, child leg
`call.transfer.failed`	Transfer fails	Transfer report
`call.conference.started`	Conference opens	Conference logs
`call.conference.participant_joined`	Someone joins	Conference participant list
`call.conference.participant_left`	Someone leaves	Conference participant list
`call.conference.ended`	Conference closes	Conference logs, CF recording
`call.forwarded`	Forwarding fires	Forwarding report, child leg

Queue and IVR

Event	When it fires	What it feeds in HRX
`queue.enqueued`	Caller enters a queue	Queue window
`queue.dequeued`	Caller leaves a queue	Queue wait time
`queue.abandoned`	Caller hangs up while queued	Abandon report
`ivr.abandoned`	Caller hangs up inside the IVR	IVR abandon report

Recordings and agent activity

Event	When it fires	What it feeds in HRX
`recording.ready`	Audio written to disk	The play button becomes active
`agent.login`	Agent signs in	Session opens
`agent.state`	Agent state changes	Wallboard, active time
`agent.break.start`	Break begins	Offtime report
`agent.break.end`	Break ends	Offtime report
`agent.logout`	Agent signs out	Session totals, productivity report

Example: queue abandon


```
{
  "event": "queue.abandoned",
  "event_id": "4d5e6f70-8192-43a4-b5c6-7d8e9f0a1b23",
  "occurred_at": "2026-08-19T15:12:31+05:30",
  "sequence": 2,
  "call_uid": "CV-20260819-000525",
  "origin": "inbound",
  "direction": "inbound",
  "caller_number": "+919812311111",
  "queue_name": "CustomerCare",
  "wait_seconds": 96,
  "abandon_point": "queue"
}
```


Example: attended transfer


```
{
  "event": "call.transfer.completed",
  "event_id": "3c4d5e6f-7081-4293-a4b5-6c7d8e9f0a12",
  "occurred_at": "2026-08-19T14:34:02+05:30",
  "sequence": 6,
  "call_uid": "CV-20260819-000481",
  "transfer": {
    "type": "attended",
    "from_agent_id": "AG-1042",
    "to_agent_id": "AG-1077",
    "to_extension": "1077",
    "transferred_at": "2026-08-19T14:34:02+05:30",
    "child_call_uid": "CV-20260819-000482",
    "consult_seconds": 18,
    "recording_uid": "REC-TL-20260819-000482"
  }
}
```

5. Identifying the agent

Two identifiers do the work. Agreeing them removes most of the integration risk.

Identifier	Generated by	Rule
`ref_id`	HRX	Sent on originate. Echo it on every event for that call and in the CDR. Present only for API-initiated calls.
`call_uid`	CentroVision	Primary key for the whole call life. Must be the same value shown as RID/UID on the CentroVision CDR screen, so both sides can look up the same call when there is a disagreement.

The agent block

For a manual dial there is no ref_id, so this block is how HRX finds the employee. Send as many of these as the gateway knows; HRX tries them in the order below.

Order	Field	Notes
1	`agent.hrms_employee_id`	The HRX employee code, e.g. EMP-096. Most reliable - please carry it if possible. Hi Tech Group will supply the mapping.
2	`agent.agent_id`	Dialer agent id, e.g. AG-1042
3	`agent.agent_code`	Agent code, e.g. mehak
4	`agent.extension`	Often the only identity a handset dial carries. Always send it if known.
5	`agent.agent_number`	The agent phone or SIM MSISDN - the leg that rings first

> If none of them resolve, the call is stored as unassigned and raised on an exceptions screen for a human to fix. It is never silently dropped: a dropped call is invisible, an unassigned one gets corrected.

6. Data dictionary

Anything CentroVision cannot supply should be flagged in writing before the scope is agreed, not discovered during acceptance testing.

timestamps

All ISO-8601 with an explicit offset. Never a naive local string, never HH:MM:SS. The gaps between these are the whole value of the CDR.

Field	Meaning
`initiated_at`	API accepted, or manual dial started. Required in practice - HRX windows the reconciliation day on this field.
`agent_ring_at`	Agent leg started ringing
`agent_answer_at`	Agent picked up
`customer_ring_at`	Customer leg started ringing
`answered_at`	Both legs bridged - talk time starts here
`ended_at`	Hangup
`wrapup_start_at / wrapup_end_at`	After-call work window
`queue_enter_at / queue_exit_at`	Queue window for inbound calls

durations

Integer seconds throughout.

Field	Meaning
`agent_ring_seconds`	How long the agent phone rang
`customer_ring_seconds`	How long the customer phone rang
`pdd_seconds`	Post-dial delay: dial to first ring. The number that exposes a poor GSM route.
`queue_wait_seconds`	Time in queue before answer or abandon
`talk_seconds`	Bridged conversation, EXCLUDING hold
`hold_seconds`	Total hold time
`wrap_seconds`	After-call work
`total_seconds`	initiated_at to ended_at
`billed_seconds`	Operator-billed pulse, for cost reporting

Optional but valuable

Object	Fields	Why it is worth sending
`quality`	mos, jitter_ms, packet_loss_percent	Settles "the line was bad" disputes with data
`cost`	billed_seconds, pulse_seconds, amount, currency	Enables cost-per-lead reporting

Object	Fields	Why it is worth sending
`gateway`	gateway_id, channel, sim_msisdn, operator, trunk	Identifies which physical GSM port and SIM carried the call
`hold_events`	Array of from, to, seconds	Per-period hold detail rather than just a total

7. Response codes and errors

Code	Meaning	Action
202	Accepted and stored	Nothing. Do not retry.
200	Duplicate - already held	Treat as success. This is the expected answer to a retry.
207	Batch processed per item	Inspect the results array.
401	Authentication failed	Read error.code. DO NOT RETRY.
403	Source address not permitted	Send Hi Tech Group the correct egress address.
413	Body over 5 MB, or over 500 events	Split and resend.
422	Missing or malformed field	Fix the payload. DO NOT RETRY.
429	Rate limit exceeded	Back off and resend.
5xx	Problem at the HRX end	Retry per the policy in section 8.

Error body

```
{
  "error": {
    "code": "invalid_signature",
    "message": "HMAC signature did not match",
    "request_id": "8f14e45f-ceea-467a-9575-9f0c1a2b3c4d"
  }
}
```

Quote request_id and Hi Tech Group can say precisely why a request was refused, rather than guessing.

Error codes you are most likely to see

error.code	Almost always means
`invalid_signature`	The JSON was re-serialised between signing and sending. Build the string once, sign it, send it.
`stale_timestamp`	The sending server clock has drifted. Run NTP.
`replayed_nonce`	A nonce was reused. It must be fresh per HTTP request, even on a retry of the same event.
`invalid_api_key`	Wrong key, or it was rotated without the new one being applied.
`ip_not_allowed`	Requests are arriving from an address Hi Tech Group has not been told about.
`missing_signature`	Header absent or not in sha256=<hex> form.
`missing_timestamp`	Header absent, or not epoch seconds.
`payload_too_large`	Body over 5 MB. Split a backlog across several batch calls.
`validation_failed`	A required field is missing. The rejected payload is stored so it can be inspected.

8. Reliability rules

Rule	Specification
Idempotency	Every event carries a unique event_id, repeated in the Idempotency-Key header. Re-delivery is safe: HRX answers 200 with status "duplicate" and does not write twice.
Ordering	Each event carries a monotonic sequence number per call_uid. HRX tolerates out-of-order arrival and never lets a late call.answered overwrite a call.completed.
Acknowledgement	HRX responds within 2 seconds. It stores the raw payload and processes it afterwards. 202 means stored, not necessarily processed.
Retry	On 5xx or timeout, retry at 1s, 5s, 30s, 2m, 10m, 1h, then hourly up to 24 hours, keeping the SAME event_id. On 4xx other than 429, do not retry - log it and escalate.
Backlog drain	After an outage, send buffered events to /events/batch, up to 500 per request, processed per item so one bad row never rejects the batch.
Time format	ISO-8601 with an explicit offset. Durations are integer seconds.
Numbers	E.164 with country code, e.g. +919876543210. If the gateway emits bare 10-digit numbers, normalise before sending.
Versioning	Path-versioned at /v1/. Additive changes only within a version; anything breaking gets /v2 with a 60-day overlap.

9. Reconciliation (pull)

HRX runs a reconciliation nightly at 02:00 IST for the previous day. It compares its own call count and total talk time against the CentroVision CDR for the same window, and backfills anything the push channel never delivered.

This is what makes the integration trustworthy rather than merely useful. Without it, "we think we are missing calls" can never be answered.

What CentroVision needs to expose

Priority	Endpoint	Purpose
1	`GET /cdr`	One page of CDR for a time window. Without this there is no audit and no recovery of lost calls.
2	`GET /recordings/{uid}/signed-url`	Short-lived (300s or less), unguessable, Range-capable URL so HRX can proxy playback safely.
3	`POST /calls/originate`	Click-to-call, returning call_uid and echoing ref_id.
4	`GET /agents`	The agent master, so the HRX mapping table can be reconciled nightly.

GET /cdr requirements

Requirement	Detail
total_count	Must be the count for the WHOLE window, not the page. HRX compares that first and only pages through records when it disagrees.
Paging	Cursor paging, 500 per page. Offset paging is acceptable if that is what exists.
Record shape	Same field set as a call.completed webhook, so a backfilled call is indistinguishable from a pushed one.
Window	An Indian business day, e.g. 2026-08-19T00:00:00+05:30 to 2026-08-19T23:59:59+05:30. Please honour the offset - a 03:00 IST call belongs to the 19th.

> HRX pulls once nightly, off-peak, paging at 500 rows, and prefers push everywhere else - so the load on the gateway stays small.

10. Acceptance checklist

Run these in order. Do not accept a stage until its test passes.

#	Test	Expected
1	Signed GET /health	200 with api_version v1
2	Deliberately wrong signature	401 invalid_signature, nothing written
3	Re-send an event already delivered	200 duplicate, not written twice
4	Malformed payload	422 validation_failed
5	One real answered call, end to end	Appears in HRX with matching call_uid and durations within 2 seconds of the CDR
6	A call dialled manually on the handset (UAT-05)	Same events, origin manual_dial. GO / NO-GO.
7	Inbound and IVR calls	Same events, origin set correctly
8	Blind and attended transfer	Transfer legs linked by child_call_uid
9	Three-way conference	Conference leg listing all participants
10	Queue abandon	status abandoned, wait_seconds populated
11	Recording playback in HRX	Audio plays, access is logged
12	Agent shift with a break	Totals match the CentroVision User Session report
13	Ten-minute HRX outage during business hours	Every call made during it arrives via retry or the nightly reconciliation
14	Seven consecutive nights of reconciliation	Zero count gap

> **The Postman collection runs tests 1 to 4 and 6 for you**
>
> HRX-Telephony.postman_collection.json contains 28 runnable requests with the signing already written, sample payloads for every event, and the negative tests. Import it, set api_key and shared_secret, and press Run. The run report is the evidence for sign-off.

Questions

Anything unclear, or any field the CentroVision platform cannot supply, should be raised now in the Field Confirmation tab of HRX-Telephony-UAT.xlsx. A field flagged at kickoff is a short conversation. The same field discovered during acceptance testing is a change request.

Nagender Kumar
Hi Tech Group